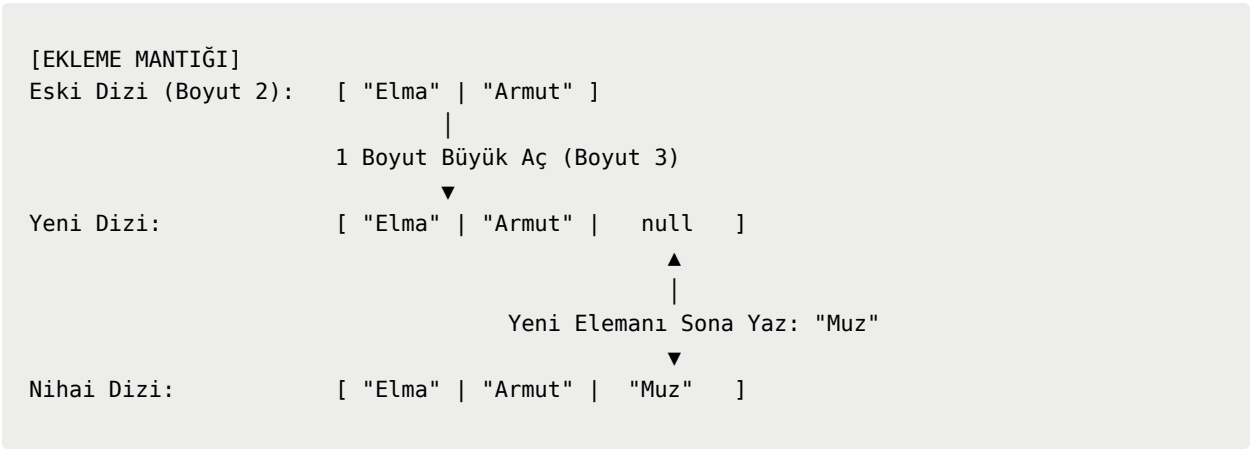
C# Ders - 26 Ekim 2026 Ders Notları

1. Dizilerin Sabit Boyut Kısıtı ve Manuel Dinamik Dizi Yönetimi

C#'ta diziler (`Array`) bellekte **sabit boyutlu (fixed-size)** olarak oluşturulur. Bir kez `new string[5]` dediğinizde o dizinin boyutu ömrü boyunca 5'tir; 6. bir eleman eklenemez veya bir eleman fiziksel olarak yok edilip boyutu 4'e düşürülemez.

Ders kodlarında yapılan işlem, ileride göreceğiniz `List<T>` koleksiyonunun arka planda nasıl çalıştığını anlamak için harika bir **algoritmik simülasyondur**.



A. Diziye Dinamik Eleman Ekleme Algoritması

Bir diziye yeni bir eleman eklemek için 3 adım izlenir:

1. Mevcut diziden **1 eleman daha büyük** geçici yeni bir dizi açılır (`new string[arr.Length + 1]`).
2. Eski dizideki tüm veriler `Array.Copy` ile bu yeni diziye aktarılır.
3. Yeni dizinin en son indeksine eklenecek yeni kelime yazılır ve bu yeni dizi geriye döndürülür.

C#

```
string[] AddElement(string kelime, string[] arr)
{
    // Eğer dizinin son hücresi henüz boşsa (null ise), yeni dizi açmaya gerek yoktur
    if (arr[arr.Length - 1] == null)
    {
        arr[arr.Length - 1] = kelime;
        return arr;
    }

    // 1. Bir büyük boyutta yeni dizi aç
    string[] newArr = new string[arr.Length + 1];

    // 2. Eski verileri yeni diziye kopyala
    Array.Copy(arr, newArr, arr.Length);

    // 3. Son göze yeni veriyi yerleştir
    newArr[arr.Length] = kelime;

    return newArr;
}
```

B. Diziden Dinamik Eleman Silme Algoritması

Bir diziden belirli bir elemanı silip boyutu küçültmek için:

1. Silinmek istenen elemanın indeksi `Array.IndexOf` ile bulunur.
2. Silinecek hücre `null` yapılarak işaretlenir.
3. Mevcut boyuttan **1 eleman daha küçük** yeni bir dizi açılır (`new string[arr.Length - 1]`).
4. `null` olan hücre **atlanarak** geriye kalan tüm dolu elemanlar yeni diziye aktarılır.

C#

```
string[] RemoveAt(int index, string[] arr)
{
    if (index < 0 || index >= arr.Length)
        return arr; // Geçersiz indeks ise işlem yapma

    // Silinecek elemanı boşa çıkar
    arr[index] = null;

    // 1 küçük boyutta yeni dizi oluştur
    string[] newArr = new string[arr.Length - 1];
    int yeniIndex = 0;
```

```
    // null OLMAYANLARI yeni diziye sıkıştırarak taşı
    for (int i = 0; i < arr.Length; i++)
    {
        if (arr[i] != null)
        {
            newArr[yeniIndex++] = arr[i];
        }
    }

    return newArr;
}
```

2. Paralel Diziler (Parallel Arrays) Mimarisi

Nesne yönelimli programlamaya (OOP / Sınıflar) geçmeden önce, birbiriyle ilişkili verileri aynı indekste tutma yöntemine **Paralel Diziler** denir.

```
İndeks No:          [0]                 [1]                 [2]
products:    [ "Ekmek (Somun)"   | "Süt (UHT Yağlı)" | "Yumurta (30'lu)" ]
prices:      [     12.50m        |      38.00m       |     145.00m       ]
sepet:       [       3           |        0          |        2          ]
                     ▲
                     |
             İlişki Mantığı:
             3 adet Ekmek alındı -> Tutar = 3 * 12.50m
```

- `products[i]` →→ ii. sıradaki ürünün adıdır.
- `prices[i]` →→ ii. sıradaki ürünün birim fiyatıdır.
- `sepet[i]` →→ Müşterinin ii. sıradaki üründen kaç adet/kilo aldığını tutar.

Neden Bu Yaklaşım Tercih Edilir?

Tek bir döngü değişkeni (`i`) ile üç farklı dizideki veriyi senkronize yönetebilirsiniz:

C#

```
for (int i = 0; i < products.Length; i++)
{
    if (sepet[i] > 0)
    {
        decimal urunToplam = sepet[i] * prices[i];
        Console.WriteLine($"{products[i]} x {sepet[i]} adet = {urunToplam:C2}");
```

```
    }
}
```

3. Yeniden Kullanılabilir Girdi Metotları (Input Helper Methods)

Kod yazarken her seferinde `Console.WriteLine()`, `Console.ReadLine()`, `int.TryParse()` kalıbını baştan yazmak kod tekrarına yol açar. Bu işlemleri amaca yönelik küçük yardımcı metotlara devretmek yazılım mimarisinin temelidir.

C#

```
// 1. Güvenli Metin Okuma (Null engelleme ile)
string MetinAl(string mesaj)
{
    Console.WriteLine(mesaj);
    return Console.ReadLine() ?? string.Empty;
}

// 2. Güvenli Tam Sayı Okuma
int IntAl(string mesaj)
{
    Console.WriteLine(mesaj);
    int.TryParse(Console.ReadLine(), out int sonuc);
    return sonuc;
}

// 3. Güvenli Ondalıklı Sayı Okuma (Para/Miktar için)
decimal DecimalAl(string mesaj)
{
    Console.WriteLine(mesaj);
    decimal.TryParse(Console.ReadLine(), out decimal sonuc);
    return sonuc;
}
```

Kod Okunabilirliğine Faydası:

C#

```
// Eski zahmetli yöntem:
Console.WriteLine("Kullanıcı adını giriniz:");
string user = Console.ReadLine();
```

```
// Yeni temiz yöntem:
string user = MetinAl("Kullanıcı adını giriniz:");
int secim = IntAl("İşlem seçiniz (1-3):");
```

4. Sonsuz Döngü (`while(true)`) ve Menü Akış Kontrolü

Menülü konsol uygulamalarında kullanıcının işlem yaptıktan sonra programdan hemen düşmemesi, istediği kadar işlem yapıp yalnızca kendi iradesiyle çıkabilmesi için `while(true)` blokları kullanılır.

Çıkış ve Akış Mekanizmaları

Komut	Kapsamı	Ne Yapar?
`break;`	Döngü veya `switch`	İçinde bulunduğu döngüyü veya switch bloğunu anında kırar, döngü sonrasındaki satırdan devam eder.
`return;`	Metot (Fonksiyon)	Çalıştığı metodun icrasını anında bitirir. Eğer `Main` içindeyse **program tamamen kapanır**.
`continue;`	Döngü	O anki turu pas geçer, döngünün bir sonraki adımına atlar.

C#

```
while (true)
{
    int islem = IntAl("1- Ekle\n2- Sil\n0- Çıkış");

    switch (islem)
    {
        case 1:
            // Ekleme işlemi...
            break; // Sadece switch'i kırar, döngü devam eder.

        case 2:
            // Silme işlemi...
            break;

        case 0:
            Console.WriteLine("Çıkış yapılıyor...");
            return; // Programı veya metodu komple sonlandırır!
```

```
        default:
            Console.WriteLine("Hatalı seçim yaptınız!");
            break;
    }
}
```

5. Güvenlik ve Bellek Temizleme: Veri Sıfırlama Mantığı

Ders kodunun sonundaki şu satır güvenlik açısından kritik bir alışkanlığı temsil eder:

C#

```
userName = string.Empty;
```

Bir oturum veya işlem tamamlandığında hafızada hassas verilerin (kullanıcı adları, şifreler, kredi kartı bilgileri) açıkta kalmasını engellemek amacıyla değişkenler `string.Empty` veya `null` yapılarak oturum izleri temizlenir.

6. Bütünsel Mimari Özeti

